Jour 1 : Fondamentaux des Réseaux de Neurones

Introduction au deep learning, architecture neuronale et optimisation

Comprendre le neurone artificiel

Maitriser les fonctions d'activation

Apprender la backpropagation

Decouvrir les optimiseurs

© 2026 UTOPIOS - Tous droits reserves

🚀 Introduction au Deep Learning

Le **Deep Learning** est une branche du Machine Learning qui utilise des reseaux de neurones artificiels pour apprendre des representations hierarchiques des donnees.

Caracteristiques du DL

- Sous-domaine du Machine Learning
- Inspire par le cerveau humain
- Apprend des representations hierarchiques
- Tres efficace pour donnees complexes

Applications principales

- Vision par ordinateur (CNN)
- Traitement du langage naturel (RNN, Transformer)
- Detection d'objets et segmentation
- Reconnaissance vocale

✅ **Point cle :** Le deep learning require de grandes quantites de donnees mais offre des performances exceptionnelles pour les taches complexes.

Le Neurone Artificiel

Le neurone artificiel est l'unite de base des reseaux de neurones. Il recoit des entrees, les traite et produit une sortie.

> **Equation du neurone :**
> $y = f(w_1x_1 + w_2x_2 + ... + w_nx_n + b)$
> où f est la fonction d'activation

Composants

- **Entrees (x)** : donnees d'entree
- **Poids (w)** : parametres appris
- **Biais (b)** : terme constant
- **Somme ponderee** : $\Sigma(w_i \cdot x_i) + b$

Fonction d'activation

- Appliquee a la somme ponderee
- Introduit la non-linearite
- Permet au reseau d'apprendre des patterns complexes
- Critiquement importante

Fonctions d'Activation

Les fonctions d'activation introduisent la non-linearite dans le reseau. Sans elles, un reseau profond se reduirait a une simple regression lineaire.

Sigmoid

```
σ(x) = 1 / (1 + e^(-x))
```

- Sortie entre 0 et 1
- Classification binaire
- Probleme : vanishing gradient

ReLU

```
f(x) = max(0, x)
```

- Tres utilisee (couches cachees)
- Rapide et efficace
- Probleme : dying ReLU

Tanh

$$\tanh(x) = (e^{2x} - 1) / (e^{2x} + 1)$$

- Sortie entre -1 et 1
- Utilisee en RNNs
- Meilleure que sigmoid

Softmax

$$\sigma(x)_j = e^{x_j} / \Sigma e^{x_i}$$

- Classification multiclasse
- Probabilites normalisees
- Couche de sortie

Fonctions de Loss (Erreur)

La loss fonction mesure l'erreur entre la prediction du modele et la vraie valeur. C'est ce que le reseau cherche a minimiser pendant l'entraînement.

MSE - Regression

$$MSE = (1/n) \Sigma (\hat{y}_i - y_i)^2$$

- Pour problemes de regression
- Penalise les grandes erreurs
- Valeurs continues

Binary Crossentropy

$$Loss = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

- Classification binaire
- Sortie : {0, 1}
- Plus efficace que MSE

Categorical Crossentropy

$$Loss = -\Sigma\ y_i \cdot \log(\hat{y}_i)$$

- Classification multiclasse (3+ classes)
- Requiert one-hot encoding
- Generalisation de binary crossentropy

↑ Backpropagation

La **backpropagation** est l'algorithme qui permet au reseau d'apprendre. Elle calcule comment chaque poids affecte l'erreur finale.

Etapes de l'entraînement

1. **Forward Pass** : calcul de la sortie $\hat{y} = f(x, w)$
2. **Calcul du Loss** : mesurer l'erreur $L = loss(\hat{y}, y)$
3. **Backward Pass** : calculer les gradients $\partial L/\partial w$ pour chaque poids
4. **Update** : $w = w - \eta \cdot \partial L/\partial w$ (descente de gradient)

⚠ **Chain Rule :** La backpropagation applique la rule de la chaine pour propaguer les erreurs de la sortie vers l'entree.

💡 **Importance :** Backpropagation rend l'entraînement de reseaux profonds efficace et pratique.

⚙️ Optimiseurs

Un optimiseur est un algorithme qui modifie les poids du reseau pour minimiser la loss function.

SGD

```
w = w - η·∂L/∂w
```

- Simple et classique
- Learning rate constant (η)
- Convergence parfois lente

Momentum

```
v = β·v + ∂L/∂w
```

- Accumule gradients passes
- Accelere convergence
- β = 0.9 typique

Adam

Combine momentum + RMSprop

- Adapte learning rate par parametre
- Standard moderne tres efficace
- Parametre par defaut : 0.001

RMSprop

Normalise par gradient magnitude

- Utile pour RNN
- Evite gradient explosion
- Learning rate adaptatif

Hyperparametres Clés

Les hyperparametres sont les variables que **nous choisissons** avant l'entraînement (contrairement aux poids qui sont appris).

Parametres importants

- **Learning Rate (η)** : controle la taille des pas. Trop eleve = divergence, trop bas = convergence lente (typique : 0.001)
- **Batch Size** : nombre d'exemples par gradient. Petit = bruyant, grand = precis mais lent (32, 64, 128 courant)
- **Epochs** : passages complets sur l'ensemble d'entraînement
- **Dropout** : pourcentage de neurones a desactiver aleatoirement (0.2 a 0.5)
- **L1/L2 Regularization** : penalite sur la magnitude des poids (evite l'overfitting)

> **Conseil :** Le tuning des hyperparametres est une art. Commencez avec des valeurs par defaut bien connues puis ajustez progressivement.

Probleme du Vanishing/Exploding Gradients

Dans les reseaux profonds, les gradients peuvent devenir tres petits (vanishing) ou tres grands (exploding) lors de la backpropagation.

Vanishing Gradient

- Gradients deviennent tres petits
- Couches profondes apprennent lentement
- Probleme avec sigmoid/tanh
- Cause : derivees < 1 propagees

Exploding Gradient

- Gradients deviennent tres grands
- Poids divergent, loss = NaN
- Instabilite numerique
- Cause : gradients > 1 accumules

Solutions

- **Utiliser ReLU** au lieu de sigmoid/tanh pour couches cachees
- **Batch Normalization** pour normaliser les activations

- **Weight initialization** minutieuse (Xavier, He initialization)
- **Gradient clipping** pour limiter l'amplitude des gradients

Resume du Module 1

Neurone Artificiel

- Entrees × poids + biais
- Activation non-lineaire
- Sortie continue

Fonctions d'Activation

- Sigmoid / Tanh classiques
- ReLU standard moderne
- Softmax pour sortie

Backpropagation

- Forward pass
- Backward pass (gradients)
- Update des poids

Points cles a retenir

- Le deep learning utilise des reseaux de neurones avec plusieurs couches
- La backpropagation permet d'apprendre efficacement les poids
- Les hyperparametres (learning rate, batch size, etc.) sont cruciaux
- Les problemes de gradient (vanishing/exploding) sont adresses par les bonnes pratiques modernes